

MQTT used in project 'Autonoom Manoeuvreren In De Haven'

Hidde Gerritsen



Studentnumber: 1079142

Course: Project 7/8

Educator: Anne de Gier & Alexander Slaa

Date: 15-5-2025

Submission: First opportunity

Contents

1 Terminology and Scope	2
2 Executive Summary	3
3 Introduction	4
3.1 Research Questions	4
4 Theoretical framework	5
4.1 MQTT: Origin and Functionality	5
4.2 Relevance in Embedded and Autonomous Systems	5
4.3 Comparison with Alternative Protocols	6
5 Methods	7
5.1 Research Steps	7
5.2 Protocol Selection Criteria	8
6 Results	9
6.1 1. What are the benefits of MQTT for real-time modular communication systems like ours?	9
6.2 2. How does MQTT compare to other communication protocols?	9
6.3 3. What are the main challenges of using MQTT in a maritime environment?	11
6.4 4. How should MQTT topics and QoS be configured to handle multiple modules?	12
7 Conclusion	14
8 References	14

1 Terminology and Scope

In this report:

- A **module** is any sensor or actuator component with its own microcontroller.
- The **central point** or **MQTT broker** is part that receives and distributes all the data from and to the modules
- The **prototype** refers to a prototype that will be the end-product for this project

2 Executive Summary

This report investigates how MQTT can be used for real-time data communication between multiple distance sensors on a ship. The goal of the project, led by Sens2Sea, is to improve the mooring process by automating ship control based on data from distributed sensor modules. MQTT was chosen as the required communication protocol.

The study shows that MQTT is well-suited for this task due to its lightweight design, low bandwidth usage, and publish/subscribe model. These features make it ideal for embedded systems with limited resources and allow for flexible, modular communication between sensors and the ship's control unit.

MQTT was compared with HTTP, WebSocket, and CAN. It offers the best balance between speed, reliability, and ease of use for real-time systems. While challenges such as harsh onboard conditions and the risk of a central broker failure exist, these can be addressed through careful design including topic structure and Quality of Service settings.

In conclusion, MQTT is a reliable and efficient protocol that supports scalable, real-time communication for autonomous maritime systems. It fits the technical needs of this project and can be used to safely and effectively control the ship's movements based on sensor data.

3 Introduction

This report has been written in response to a specific requirement set by our project developer, which states: *"The prototype must use MQTT as the data transfer protocol."*

The project is led by Geert Mosterdijk, an employee at **Sens2Sea**, a company specialising in radar and maritime measurement technology.

Currently, the mooring of ships in harbours is largely carried out by the visual judgement of the captain, supported by sailors and a limited number of sensors placed around the ship. This manual process is both inefficient and error-prone. Human error can lead to excessive throttle use, unnecessary energy consumption, and increased operational costs due to the need for manual labour.

Considering the shipping industry moved 11.61 billion tons of cargo in 2023 [1], even a small improvement in the mooring process can lead to significant economic and environmental gains. Automating this process would not only reduce human error but also increase operational efficiency.

To realise this, Geert Mosterdijk envisions a system where multiple sensors are placed around the ship. These sensors will send real-time data to a central unit, which will then control the ship's engines and propellers accordingly. The communication between the sensors and the central unit will be handled using the MQTT protocol, the core subject of this report.

3.1 Research Questions

"How can MQTT be applied to transfer data between different distance sensing modules?"

To answer this question, the report is divided into several sub-questions, each addressing a key aspect of MQTT and its suitability for our use case:

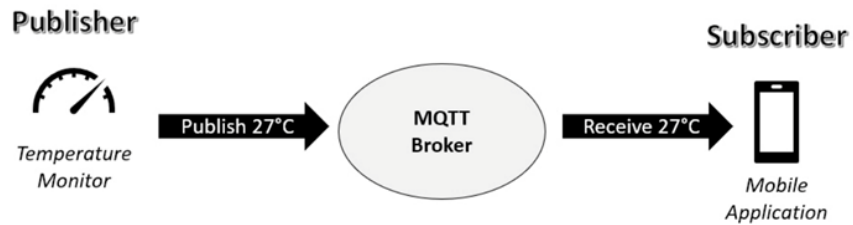
1. What are the benefits of MQTT for real-time modular communication systems like ours?
2. How does MQTT compare to other communication protocols?
3. What are the main challenges of using MQTT in a maritime environment?
4. How should MQTT topics and QoS be configured to handle multiple modules?

4 Theoretical framework

To understand how MQTT can be applied in our maritime prototype, it is important to explore the origin, purpose, and current usage of this protocol, as well as relevant alternatives and contextual factors.

4.1 MQTT: Origin and Functionality

MQTT (Message Queuing Telemetry Transport) was developed in 1999 by IBM for use in remote oil pipelines with unreliable connections. It is a lightweight messaging protocol designed for constrained devices and low-bandwidth, high-latency networks. It uses a **publish/subscribe** model, where clients send messages to a **broker**, which then forwards those messages to other clients subscribed to specific **topics**.



This architecture decouples senders and receivers, making the system modular, scalable, and suitable for real-time distributed systems like ours [2].

4.2 Relevance in Embedded and Autonomous Systems

MQTT is widely used in:

- **home automation**, MQTT is favored for its lightweight nature and low bandwidth usage, which makes it ideal for connecting simple devices like smart lights, thermostats, and door sensors. Its publish/subscribe model allows seamless communication between multiple devices and central controllers, often through platforms like Home Assistant or cloud brokers.
- **industrial IoT**, MQTT plays a crucial role due to its ability to handle high volumes of sensor data across large-scale deployments. Factories use MQTT to transmit telemetry data such as temperature, vibration, and pressure from machines to monitoring systems in real time. Its support for

Quality of Service (QoS) levels ensures reliable message delivery, which is essential in safety-critical environments.

- **autonomous robotics**, MQTT enables modular system design by decoupling the communication between components such as navigation units, sensors, and actuators. Robots operating in dynamic environments benefit from the protocol's ability to manage asynchronous messaging, allowing various modules to work together efficiently without requiring constant direct connections.

[3]

4.3 Comparison with Alternative Protocols

MQTT is not the only communication protocol used in embedded systems. Below are three other common options and how they compare to MQTT:

- **HTTP** is the standard protocol used for websites. It is called "heavy" because it needs a lot of extra data (headers, handshakes) to send even a small message. It also opens and closes a connection every time, which makes it slower and uses more energy. In contrast, MQTT keeps a connection open and is better for sending small, fast messages between sensors.
- **WebSocket** allows real-time two-way communication. It keeps the connection open, so messages can go back and forth quickly. This works well for live data like video or chat. However, it uses more memory and energy than MQTT, and it does not have built-in features like Quality of Service (QoS) or message buffering when the connection is lost. MQTT is better suited for devices with limited power or slow networks.
- **CAN (Controller Area Network)** is often used in cars and machines. It is very fast and reliable, but only works over short distances and requires special hardware. It is not designed for wireless use. MQTT works over Wi-Fi or mobile networks, making it better for wireless communication between devices.

In summary, MQTT is more lightweight and reliable. It handles many different data inputs well, uses less bandwidth, works well on small devices and is easy to scale for larger systems. [4]

5 Methods

This report is based on information coming from literature research only, aiming to answer all the main- and sub-questions as written in the introduction.

5.1 Research Steps

To answer the main and sub-questions, the following steps were taken:

1. Literature Review on MQTT

To address sub-question 1, MQTT documentation [2], and technical blogs were used to explain the core concepts and benefits of MQTT, especially in embedded or IoT contexts.

2. Comparison with Alternative Protocols

Sub-question 2 was answered through comparative analysis. Key sources [4] were used to evaluate HTTP, WebSocket, and CAN against MQTT based on technical parameters like latency, bandwidth, and scalability.

3. Challenges and Configuration

To answer sub-question 3, sources describing MQTT implementation in embedded systems [5] were analysed. These included studies on QoS, topic structure, network limitations, and security models.

4. Application in Related Systems

To address sub-question 4, existing use cases of MQTT in similar systems — such as autonomous vehicles, industrial automation, and robotic platforms — were explored to draw relevant insights.

5.2 Protocol Selection Criteria

To systematically compare MQTT with other communication protocols, the following evaluation criteria were used:

Criteria	Explanation
Latency	Is the protocol suitable for fast, low-delay communication?
Reliability	Can it guarantee message delivery?
Scalability	Can it support multiple modules?
Bandwidth Efficiency	How much data overhead does the protocol add?
Complexity	How difficult is the implementation on embedded hardware?

Table 1: Evaluation criteria for comparing MQTT with other communication protocols. [4]

6 Results

This section presents the findings resulting from the literature study and theoretical analysis, directly addressing each of the sub-questions introduced earlier.

6.1 1. What are the benefits of MQTT for real-time modular communication systems like ours?

MQTT offers multiple benefits that make it ideal for modular, real-time systems such as autonomous maritime platforms:

- **Low Bandwidth Usage:** MQTT is a lightweight protocol that minimizes data overhead, which is particularly useful in embedded systems with limited network capacity [2].
- **Asynchronous Communication:** Through its publish/subscribe model, MQTT decouples sender and receiver, allowing modules to function independently and improving system robustness.
- **Persistent Sessions:** MQTT supports persistent connections, which reduce the overhead of repeated handshakes and ensure continuity in data transmission even after temporary disconnections.
- **Quality of Service (QoS):** MQTT includes configurable QoS levels (0, 1, 2), enabling developers to balance speed versus reliability depending on the importance of each message [5].
- **Lightweight Client Implementation:** MQTT clients can be implemented using minimal code and memory, which is suitable for microcontroller-based sensor modules.

6.2 2. How does MQTT compare to other communication protocols?

Based on the criteria defined in the methodology section, MQTT was compared to HTTP, WebSocket, and CAN bus. The evaluation focused on latency, reliability, bandwidth usage, suitability for embedded systems, and scalability.

- **Latency:** MQTT generally exhibits low latency due to its lightweight nature and persistent TCP connections. WebSocket also performs well in terms of latency thanks to its full-duplex communication model. HTTP, on the other hand, suffers from high latency due to the overhead of repeated handshakes and header transmission. CAN is extremely low-latency in local wired networks but is not suitable for wide-area or internet-connected systems [4].
- **Reliability:** MQTT provides three Quality of Service (QoS) levels (0, 1, 2), allowing developers to guarantee delivery where necessary. WebSocket and HTTP do not provide built-in message delivery guarantees. CAN bus offers reliable delivery in small-scale local networks, but lacks mechanisms for confirming delivery over larger systems [5].
- **Bandwidth Usage:** MQTT has minimal overhead, typically requiring just a few bytes per message, making it highly efficient for low-bandwidth environments. HTTP is the most bandwidth-intensive due to its verbose headers and connection model. WebSocket is moderately efficient, with low per-message overhead once a connection is established. CAN bus is optimized for short, fixed-length messages but operates over wired networks and does not scale efficiently in terms of bandwidth across multiple nodes.
- **Suitability for Embedded Systems:** MQTT clients are lightweight and easy to implement on constrained devices with limited CPU and memory. HTTP requires more resources due to its text-based protocol and repeated handshakes. WebSocket clients are lighter than HTTP but still require a continuous TCP connection and more RAM. CAN bus is ideal for embedded systems in vehicles or machinery but requires specific hardware and is not IP-based.
- **Scalability:** MQTT is highly scalable due to its topic-based publish/subscribe model, allowing for dynamic addition of clients without direct reconfiguration of others. HTTP and WebSocket follow a client/server model, which does not scale as efficiently for large, distributed systems. CAN is limited to a fixed number of devices per bus and does not support wide-area communication.

This comparison demonstrates that MQTT strikes the best balance across all selected criteria. Its low latency, reliability features, and low resource usage make it

ideal for modular embedded systems with real-time communication requirements, such as the autonomous sensor platform developed in this project.

6.3 3. What are the main challenges of using MQTT in a maritime environment?

In the context of this project, MQTT is used to transmit sensor data from a wired onboard module to a central broker on the same vessel. This data is then processed to support autonomous or assisted ship control. While the maritime environment is often associated with wireless and satellite communications, this setup avoids such challenges by relying on a local wired Ethernet network. Nonetheless, the maritime context still introduces specific challenges:

- **Harsh environmental conditions:** Onboard systems are exposed to vibrations, temperature fluctuations, humidity, and salt. These can affect Ethernet cabling and connectors, leading to possible data loss or degraded physical-layer performance. MQTT relies on persistent TCP connections, so physical disconnections may interrupt data streams unless recovery strategies are in place.
- **Real-time requirements:** Sensor data may be used for time-critical control decisions, such as adjusting propulsion or rudder angle. Although MQTT supports low-latency communication, its publish-subscribe model introduces variability in message timing, especially when using higher QoS levels or broker-side buffering.
- **Broker as a single point of failure:** The central broker becomes mission-critical. If the broker fails, sensor data cannot be routed, potentially disrupting control systems. Broker redundancy or fallback mechanisms are therefore essential in maritime safety-critical applications.
- **Resource limitations on embedded modules:** The onboard sensor module may run on constrained hardware with limited CPU and memory. MQTT is lightweight, but depending on the frequency and structure of messages, even minimal overhead may accumulate—especially when multiple sensor channels operate simultaneously.

While MQTT is well-suited for this application due to its efficiency and scalability, ensuring robustness and real-time reliability in a dynamic onboard environment remains a critical design focus.

6.4 4. How should MQTT topics and QoS be configured to handle multiple modules?

In this project, multiple sensor modules onboard the ship send real-time data to a central MQTT broker for autonomous control. Proper configuration of MQTT topics and Quality of Service (QoS) levels is essential to ensure organized, reliable, and efficient data communication across all modules.

Topic Structure A clear and hierarchical topic structure improves scalability and maintainability. For multiple modules, a recommended approach is to use topics with distinct prefixes representing each sensor or module, for example:

```
ship/sensors/module1/distance
ship/sensors/module2/distance
ship/sensors/module3/distance
```

This structure allows:

- **Easy subscription management:** The central controller can subscribe to `ship/sensors+/distance` to receive data from all modules without subscribing to each separately.
- **Module-specific control:** Commands or configuration updates can be sent by publishing to module-specific topics, e.g., `ship/commands/module1`.
- **Clear data separation:** Prevents data collisions or confusion between modules.

Quality of Service (QoS) MQTT defines three QoS levels, each balancing delivery assurance and network/resource overhead:

- **QoS 0 (At most once):** Messages are sent once without confirmation. This is the lightest option but risks data loss. Suitable for non-critical sensor data or high-frequency updates where occasional loss is acceptable.
- **QoS 1 (At least once):** Ensures messages are delivered at least once but may cause duplicates. Suitable for most sensor data where reliability is important but duplicates can be handled by the application.

- **QoS 2 (Exactly once):** Guarantees exactly one delivery per message. It has the highest overhead and latency, generally reserved for critical control commands or configuration messages.

Given the maritime environment and the need for reliable sensor data for autonomous control, a mix of QoS levels can be applied:

- Sensor data topics (e.g., `ship/sensors/moduleX/distance`) should use **QoS 1** to balance reliability and efficiency.
- Command or configuration topics (e.g., `ship/commands/moduleX`) should use **QoS 2** to ensure critical instructions are received exactly once.
- Less critical telemetry or heartbeat messages can use **QoS 0** to reduce overhead.

Additional Considerations

- **Retained messages:** For topics where the latest state must be known immediately by new subscribers (e.g., module status), using retained messages can improve system responsiveness.
- **Message payload format:** Consistent and compact payload encoding (e.g., JSON, binary) reduces bandwidth and parsing complexity.
- **Topic access control:** Implementing strict topic-based authentication and authorization prevents unauthorized devices from publishing or subscribing to topics, enhancing security in the maritime environment.
- **Broker capacity:** The broker must efficiently handle simultaneous connections and message throughput from all modules without introducing delays or message loss.

In summary, a hierarchical topic naming scheme combined with appropriate QoS levels tailored to message criticality ensures reliable, scalable, and maintainable MQTT communication for multiple sensor modules onboard the ship.

7 Conclusion

This report has investigated how MQTT can be applied to exchange data between multiple distance sensor modules in the context of autonomous maneuvering in a harbor environment. The main research question was: *"How can MQTT be used to facilitate data communication between different distance sensor modules?"*

Based on the theoretical analysis and practical considerations, it is evident that MQTT is highly suitable for this application. MQTT's lightweight publish/subscribe communication model enables efficient and scalable data transfer between distributed sensor modules and the central control unit. This architecture allows sensor modules to independently publish their measurements to designated topics, while subscribers can receive only the data relevant to them, promoting modularity and flexibility.

Furthermore, MQTT's Quality of Service (QoS) levels provide configurable reliability options, which are crucial in ensuring timely and dependable data delivery in maritime environments where network conditions may be unstable. Compared to alternative communication protocols such as HTTP and WebSocket, MQTT offers a better balance between bandwidth efficiency, latency, and implementation complexity, making it ideal for embedded sensor networks.

However, the dependence on a centralized MQTT broker introduces a single point of failure, which should be addressed by implementing broker redundancy or failover mechanisms to enhance system robustness.

In conclusion, MQTT effectively meets the communication requirements for exchanging data between distance sensor modules in this project, offering a reliable, efficient, and scalable solution that supports the autonomous operation of maritime vehicles.

8 References

References

- [1] Fortune Business Insights. "Cargo shipping market size, share & covid-19 impact analysis, by cargo type..." Accessed: May 15, 2025. (2024), [Online]. Available: <https://www.fortunebusinessinsights.com/cargo-shipping-market-102045>.

- [2] MQTT.org. “What is mqtt?” Accessed: May 15, 2025. (2025), [Online]. Available: <https://mqtt.org/faq/>.
- [3] R. Morabito, V. Cozzolino, A. Iera, and G. Ruggeri, “The use of MQTT in M2M and IoT systems: A survey,” *IEEE Access*, vol. 8, pp. 201 071–201 086, 2020. DOI: 10.1109/ACCESS.2020.3037252. [Online]. Available: <https://ieeexplore.ieee.org/document/9253113>.
- [4] N. Naik, “Choice of effective messaging protocols for iot systems: Mqtt, coap, amqp and http,” *IEEE International Systems Engineering Symposium (ISSE)*, pp. 1–7, 2017. [Online]. Available: https://pure.port.ac.uk/ws/portalfiles/portal/12197128/IoT_Messaging_Protocols_Naik.pdf.
- [5] HiveMQ. “Mqtt essentials: A lightweight iot protocol.” Accessed: May 15, 2025. (2025), [Online]. Available: <https://www.hivemq.com/mqtt-essentials/>.